

4.0 Logical Physical Layer

The Logical PHY comprehends the following functions:

- Link initialization, training and power management states
- Byte to Lane mapping for data transmission over Lanes
- Interconnect redundancy remapping (when required)
- Transmitting and receiving sideband messages
- Scrambling and training pattern generation
- Lane reversal
- Width degradation (when applicable)

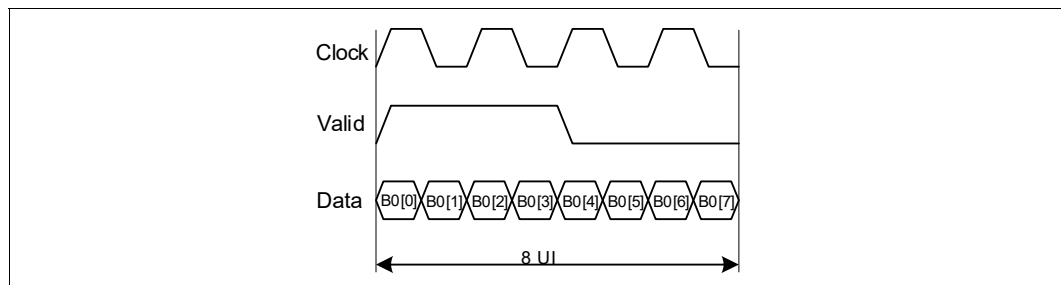
4.1 Data and Sideband Transmission Flow

This specification defines clock, valid and Data to send and receive data over the physical Lanes. The transmitted data is framed by the valid signal.

4.1.1 Byte to Lane Mapping

Data packets are transmitted in Bytes. Within each Byte, bit [0] is transmitted first. [Figure 4-1](#) shows an example of bit arrangement within one byte transmission over Lane 0.

Figure 4-1. Bit arrangement within a byte transfer



Each Byte is transmitted on a separate Lane. Byte 0 (B0) is transmitted on Lane 0, Byte 1 is transmitted on Lane 1 and so on.

[Figure 4-2](#) shows an example of a 256B Flit transmitted over a x64 interface (one x64 Advanced Package module or two x32 Advanced Package modules or four Standard Package modules). If the I/O width changes to x32 or x16 interface (Standard Package), transmission of one Byte per Lane is preserved as shown in [Figure 4-3](#) and [Figure 4-4](#) respectively.

[Figure 4-5](#) shows an example for a width degraded Standard Package module.

Figure 4-2. Byte map for x64 interface

UI \ Lane	0	1	2	3	4	5	6	7	...	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63
0 - 7	B00	B01	B02	B03	B04	B05	B06	B07	...	B48	B49	B50	B51	B52	B53	B54	B55	B56	B57	B58	B59	B60	B61	B62	B63
8 - 15	B64	B65	B66	B67	B68	B69	B70	B71	...	B112	B113	B114	B115	B116	B117	B118	B119	B120	B121	B122	B123	B124	B125	B126	B127
16 - 23	B128	B129	B130	B131	B132	B133	B134	B135	...	B176	B177	B178	B179	B180	B181	B182	B183	B184	B185	B186	B187	B188	B189	B190	B191
24 - 31	B192	B193	B194	B195	B196	B197	B198	B199	...	B240	B241	B242	B243	B244	B245	B246	B247	B248	B249	B250	B251	B252	B253	B254	B255

Figure 4-3. Byte map for x32 interface

UI \ Lane	0	1	2	3	4	5	6	7	...	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
0 - 7	B0	B1	B2	B3	B4	B5	B6	B7	...	B16	B17	B18	B19	B20	B21	B22	B23	B24	B25	B26	B27	B28	B29	B30	B31
8 - 15	B32	B33	B34	B35	B36	B37	B38	B39	...	B48	B49	B50	B51	B52	B53	B54	B55	B56	B57	B58	B59	B60	B61	B62	B63
16 - 23	B64	B65	B66	B67	B68	B69	B70	B71	...	B80	B81	B82	B83	B84	B85	B86	B87	B88	B89	B90	B91	B92	B93	B94	B95
24 - 31	B96	B97	B98	B99	B100	B101	B102	B103	...	B112	B113	B114	B115	B116	B117	B118	B119	B120	B121	B122	B123	B124	B125	B126	B127
32 - 39	B128	B129	B130	B131	B132	B133	B134	B135	...	B144	B145	B146	B147	B148	B149	B150	B151	B152	B153	B154	B155	B156	B157	B158	B159
40 - 47	B160	B161	B162	B163	B164	B165	B166	B167	...	B176	B177	B178	B179	B180	B181	B182	B183	B184	B185	B186	B187	B188	B189	B190	B191
48 - 55	B192	B193	B194	B195	B196	B197	B198	B199	...	B208	B209	B210	B211	B212	B213	B214	B215	B216	B217	B218	B219	B220	B221	B222	B223
56 - 63	B224	B225	B226	B227	B228	B229	B230	B231	...	B240	B241	B242	B243	B244	B245	B246	B247	B248	B249	B250	B251	B252	B253	B254	B255

Figure 4-4. Byte map for x16 interface

UI \ Lane	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0 - 7	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12	B13	B14	B15
8 - 15	B16	B17	B18	B19	B20	B21	B22	B23	B24	B25	B26	B27	B28	B29	B30	B31
16 - 23	B32	B33	B34	B35	B36	B37	B38	B39	B40	B41	B42	B43	B44	B45	B46	B47
24 - 31	B48	B49	B50	B51	B52	B53	B54	B55	B56	B57	B58	B59	B60	B61	B62	B63
32 - 39	B64	B65	B66	B67	B68	B69	B70	B71	B72	B73	B74	B75	B76	B77	B78	B79
40 - 47	B80	B81	B82	B83	B84	B85	B86	B87	B88	B89	B90	B91	B92	B93	B94	B95
48 - 55	B96	B97	B98	B99	B100	B101	B102	B103	B104	B105	B106	B107	B108	B109	B110	B111
56 - 63	B112	B113	B114	B115	B116	B117	B118	B119	B120	B121	B122	B123	B124	B125	B126	B127
64 - 71	B128	B129	B130	B131	B132	B133	B134	B135	B136	B137	B138	B139	B140	B141	B142	B143
72 - 79	B144	B145	B146	B147	B148	B149	B150	B151	B152	B153	B154	B155	B156	B157	B158	B159
80 - 87	B160	B161	B162	B163	B164	B165	B166	B167	B168	B169	B170	B171	B172	B173	B174	B175
88 - 95	B176	B177	B178	B179	B180	B181	B182	B183	B184	B185	B186	B187	B188	B189	B190	B191
96 - 103	B192	B193	B194	B195	B196	B197	B198	B199	B200	B201	B202	B203	B204	B205	B206	B207
104 - 111	B208	B209	B210	B211	B212	B213	B214	B215	B216	B217	B218	B219	B220	B221	B222	B223
112 - 119	B224	B225	B226	B227	B228	B229	B230	B231	B232	B233	B234	B235	B236	B237	B238	B239
120 - 127	B240	B241	B242	B243	B244	B245	B246	B247	B248	B249	B250	B251	B252	B253	B254	B255

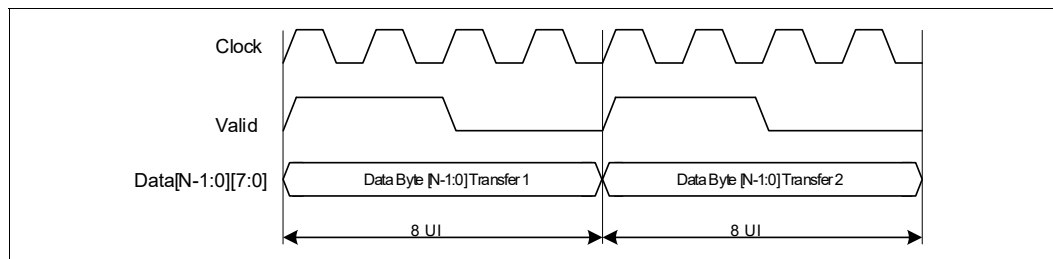
Figure 4-5. Byte to Lane mapping for Standard package x16 degraded to x8

Lane	8	9	10	11	12	13	14	15
or								
UI \ Lane	0	1	2	3	4	5	6	7
0 - 7	B0	B1	B2	B3	B4	B5	B6	B7
8 - 15	B8	B9	B10	B11	B12	B13	B14	B15
16 - 23	B16	B17	B18	B19	B20	B21	B22	B23
24 - 31	B24	B25	B26	B27	B28	B29	B30	B31
32 - 39	B32	B33	B34	B35	B36	B37	B38	B39
40 - 47	B40	B41	B42	B43	B44	B45	B46	B47
48 - 55	B48	B49	B50	B51	B52	B53	B54	B55
56 - 63	B56	B57	B58	B59	B60	B61	B62	B63
64 - 71	B64	B65	B66	B67	B68	B69	B70	B71
72 - 79	B72	B73	B74	B75	B76	B77	B78	B79
80 - 87	B80	B81	B82	B83	B84	B85	B86	B87
88 - 95	B88	B89	B90	B91	B92	B93	B94	B95
96 - 103	B96	B97	B98	B99	B100	B101	B102	B103
104 - 111	B104	B105	B106	B107	B108	B109	B110	B111
112 - 119	B112	B113	B114	B115	B116	B117	B118	B119
120 - 127	B120	B121	B122	B123	B124	B125	B126	B127
128-135	B128	B129	B130	B131	B132	B133	B134	B135
136-143	B136	B137	B138	B139	B140	B141	B142	B143
144-151	B144	B145	B146	B147	B148	B149	B150	B151
152-159	B152	B153	B154	B155	B156	B157	B158	B159
160-167	B160	B161	B162	B163	B164	B165	B166	B167
168-175	B168	B169	B170	B171	B172	B173	B174	B175
176-183	B176	B177	B178	B179	B180	B181	B182	B183
184-191	B184	B185	B186	B187	B188	B189	B190	B191
192-199	B192	B193	B194	B195	B196	B197	B198	B199
200-207	B200	B201	B202	B203	B204	B205	B206	B207
208-215	B208	B209	B210	B211	B212	B213	B214	B215
216-223	B216	B217	B218	B219	B220	B221	B222	B223
224-231	B224	B225	B226	B227	B228	B229	B230	B231
232-239	B232	B233	B234	B235	B236	B237	B238	B239
240-247	B240	B241	B242	B243	B244	B245	B246	B247
248-255	B248	B249	B250	B251	B252	B253	B254	B255

4.1.2 Valid Framing

Valid signal is used to frame the transmitted data. For each 8-bit data packet, valid is asserted for the first 4 UI and de-asserted for 4 UI. This will allow data transfer in Raw Format or various Flit Formats as described in [Chapter 3.0](#) using one or multiple valid frames. An example is shown in [Figure 4-6](#) where Transfer 1 and Transfer 2 can be from the same Flit or different Flits.

Note: An 8-UI block assertion is enforced by the Transmitter and tracked by the Receiver during Active state. This means that following the first valid transfer of data over mainband in Active state, each subsequent transfer is after an integer multiple of 8 UI from the rising edge of Valid of the first transfer. Note that for Retimers, this means that the first transfer after entering the Active state cannot be a 'No Flit data transfer + 1 credit release' encoding; this is acceptable because the Retimer-advertised credits are replenished or readvertised whenever the state moves away from Active.

Figure 4-6. Valid framing example


4.1.2.1 Valid Framing for Retimers

The UCIE Retimer releases credits to its local UCIE die using the Valid wire, as described in Table 4-1. Each credit tracks 256 Bytes of data (including any FEC, CRC, etc). The Valid Framing encodings ensure triple bit flip detection guarantee. It is permitted for receiver implementations to trigger Retrain on any bit error (using triple bit flip detection guarantee). It is also permitted for receiver implementations to use the encodings below to correct single bit errors (with the understanding that three bit error detection is lost, and its contribution to overall FIT is negligible).

Table 4-1. Valid framing for Retimers

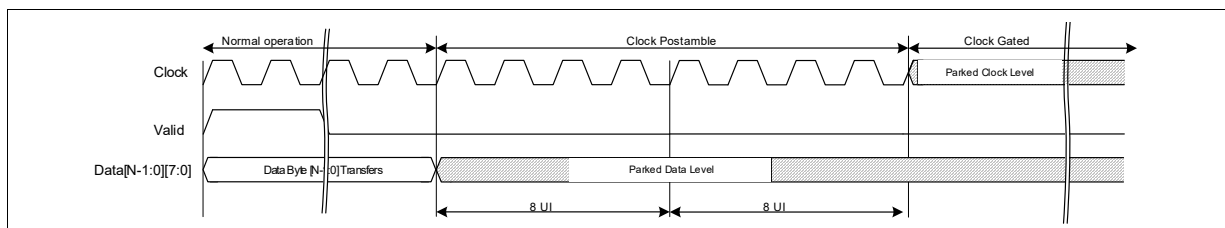
8-UI Valid ^a	Encoding
00000000b	No Flit data transfer + no credit release
00001111b	Flit data transfer valid + no credit release
11110000b	No Flit data transfer + 1 credit release
11111111b	Flit data transfer valid + 1 Credit release

a. Note that the bits above are transmitted on the Link in order from right to left (i.e., bit 0 is transmitted on the Link first, followed by bit 1 and so on until bit 7).

4.1.3 Clock Gating

Forwarded mainband clocks on the UCIE Link must be gated when Valid signal is low after providing fixed 16 UI (8 cycles) of postamble clock for half-rate clocking and 32 UI (8 cycles) of postamble clock for quarter-rate clocking, unless free running clock mode is negotiated or Runtime Recalibration has been requested by the remote Link partner. Data and Clock signal parking levels when clocks are gated are described in Section 5.11.

Note that the clock postamble is required any time that the clock can toggle with Valid assertion, and the clock needs to stop toggling, regardless of LTSM state.

Figure 4-7. Clock gating


4.1.4 Free Running Clock Mode

Free running clock mode is defined as the mode where the forwarded clock remains toggling even when the Transmitter for Valid is held low and there is no data transfer on the interface. This mode must be supported to allow disabling dynamic clock gating for normal operation or debug. This must be negotiated prior to mainband Link training through parameter exchange.

4.1.5 Sideband transmission

Each module supports a sideband interface with a serial data and clock pin pair. Sideband packet formats and encodings are shown in [Chapter 7.0](#) and the electrical characteristics are shown in [Section 5.13](#).

As shown in [Section 7.1.2](#), the sideband message formats are defined as a 64-bit header with no data, with 32 bits or 64 bits of data. A 64-bit serial packet is defined on the I/O interface to the remote die as shown in [Figure 4-8](#). 32-bit data is sent using the 64-bit serial packet with MSBs padded with 0b. Two sideband serial packets on the I/O interface are separated by a minimum of 32 bits low as shown in [Figure 4-9](#). A sideband message with data would be transferred as a 64-bit header followed by 32 bits of low followed by 64-bit data followed by 32 bits of low.

Figure 4-8. Example 64-bit Sideband Serial Packet Transfer

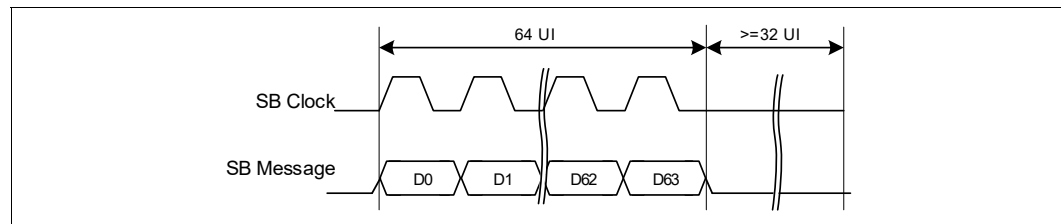
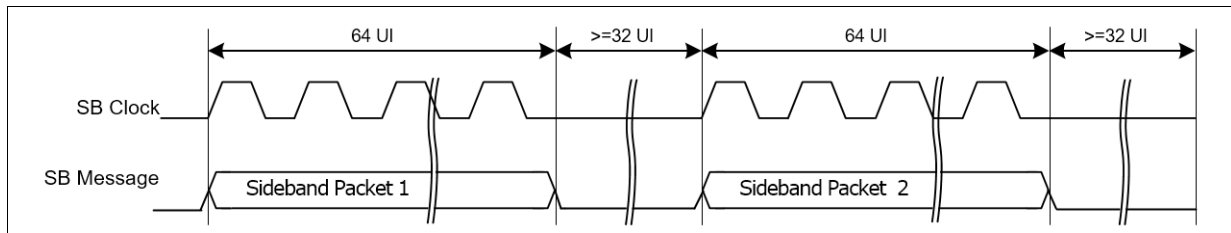


Figure 4-9. Sideband Packet Transmission: Back-to-Back



4.1.5.1 Sideband Performant Mode Operation (PMO)

Sideband designs can negotiate support for 'Sideband Performant Mode Operation (PMO)' by way of the Sideband Feature Extensions (SBFE) mechanism defined in [Section 4.5.3.3.1.1](#). The Sideband PMO bit of the {MBINIT.PARAM SBFE req/resp} sideband message (bit 1) is defined for negotiating this operation (see [Table 7-11](#)). When supporting this feature, a UCIE Link must support this capability on both its transmit and receive direction or not support the capability at all. In a multi-module Link, all modules must advertise the same capability. A UCIE Link must set the Sideband PMO bit to 1 on the {MBINIT.PARAM SBFE resp} sideband message that the UCIE Link transmits, but only if the corresponding Req message also has that bit set to 1 and its receiver is capable of Sideband PMO. Otherwise, the Sideband PMO bit must be cleared to 0.

When Sideband PMO capability is enabled, the 32-UI dead time between the 64-UI data transfers on the sideband is no longer applicable and the sideband can transmit 64-UI data back-to-back with no gaps. See [Figure 4-10](#) and [Figure 4-11](#) for illustration. The transmitter must follow this new mode after the transmitter has sent and received the {MBINIT.PARAM SBFE resp} sideband message with

the Sideband PMO bit set to 1, across all modules. The receiver must be ready to accept packets in this mode after the receiver has transmitted the {MBINIT.PARAM SBFE resp} sideband message with the Sideband PMO bit set to 1. After Sideband PMO is enabled, the transmitter operates in Performant Mode in all states until entry into the RESET state with the SB_MGMT_UP flag cleared to 0. Additionally, PMO can then only be renegotiated on a training sequence with the SB_MGMT_UP flag cleared to 0. Note that from a receiver perspective, due to timing differences, packets might be received without the Sideband Performant Mode even after the chiplet has transmitted an {MBINIT.PARAM SBFE resp} sideband message with the PMO bit set to 1. The sideband receiver must be backward compatible and be able to handle 32 UI of gaps between consecutive 64-UI transfers over the sideband Link.

Figure 4-10. Example 64-bit Sideband Serial Packet Transfer in Sideband Performant Mode

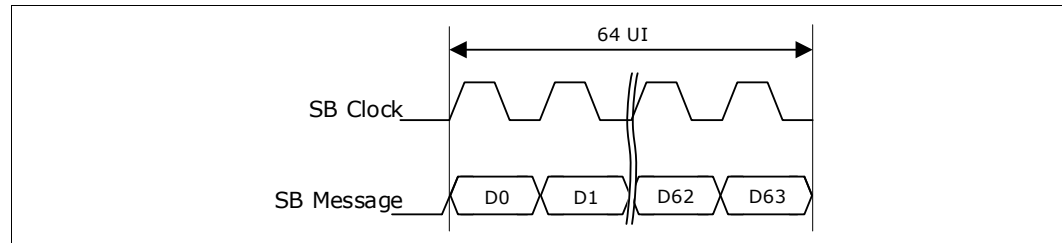
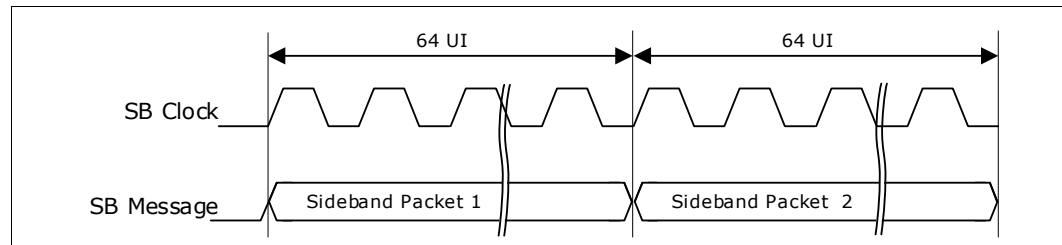


Figure 4-11. Sideband Packet Transmission: Back-to-Back in Sideband Performant Mode



4.2 Lane Reversal

In [Section 4.2](#), [Section 4.3](#), and [Section 4.5](#), the following nomenclature is used:

- **TD_P**: Physical Lane for Data Transmitter
- **RD_P**: Physical Lane for Data Receiver
- **TRD_P**: Physical Lane for Redundant Data Transmitter
- **RRD_P**: Physical Lane for Redundant Data Receiver
- **TD_L**: Logical Lane for Data Transmit
- **RD_L**: Logical Lane for Data Receive
- **TCKP_P**, **TCKN_P** and **TTRK_P**: Physical Lane for Clock and Track Transmitter
- **TCKP_L**, **TCKN_L** and **TTRK_L**: Logical Lane for Clock and Track Transmitter
- **RCKP_P**, **RCKN_P** and **RTRK_P**: Physical Lane for Clock and Track Receiver
- **RCKP_L**, **RCKN_L** and **RTRK_L**: Logical Lane for Clock and Track Receiver
- **TRDCK_P**: Physical Lane for Redundant Clock/Track Transmitter
- **RRDCK_P**: Physical Lane for Redundant Clock/Track Receiver

- **TRDCK_L**: Logical Lane for Redundant Clock/Track Transmitter
- **RRDCK_L**: Logical Lane for Redundant Clock/Track Receiver
- **TVLD_P**, **RVLD_P**: Physical Lane for Valid Transmitter and Receiver
- **TRDVLD_P**, **RRDVLD_P**: Physical Lane for Redundant Valid Transmitter and Receiver
- **TVLD_L**, **RVLD_L**: Logical Lane for Valid Transmitter and Receiver
- **TRDVLD_L**, **RRDVLD_L**: Logical Lane for Redundant Valid Transmitter and Receiver

Devices must support Lane reversal of data Lanes within a Module. An example of Lane reversal is when physical Data Lane 0 on local die is connected to physical Data Lane (N-1) on the remote die (physical Data Lane 1 is connected to physical Data Lane N-2 and so on) where N = 8 for a x8 Standard Package, N = 16 for a x16 Standard Package, N = 32 for a x32 Advanced Package, and N = 64 for a x64 Advanced Package. Redundant Lanes, in case of Advanced Package, are also reversed. Lane reversal must be implemented on the Transmitter only. The Transmitter reverses the logical Lane order on Data and Redundant Lanes.

Track, Valid, Clock, and sideband signals must not be reversed.

Lane reversal is discovered and applied during initialization and training (see [Section 4.5.3.3.5](#)).

4.2.1 Lane ID

To allow Lane reversal discovery, each logical Data and redundant Lane within a module is assigned a unique Lane ID. The assigned Lane IDs are shown in [Table 4-2](#) and [Table 4-3](#) for Advanced and Standard Package modules, respectively. Note that logical Lane numbers in [Table 4-2](#) and [Table 4-3](#) represent the logical Transmitter and Receiver Lanes. For example, Logical Lane Number = 0 represents **TD_L[0]/RD_L[0]** and so on.

In [Table 4-2](#), for a x64 Advanced Package module, logical Lane numbers 64, 65, 66, and 67 represent Logical redundant Lanes **TRD_L[0]/RRD_L[0]**, **TRD_L[1]/RRD_L[1]**, **TRD_L[2]/RRD_L[2]**, **TRD_L[3]/RRD_L[3]**, respectively. For a x32 Advanced Package module, the Lane ID for **TD_L[0:31]/RD_L[0:31]**, **TRD_L[0]/RRD_L[0]** and **TRD_L[1]/RRD_L[1]** will be represented by the set of Lane ID {0...31, 64, 65} respectively.

In [Table 4-3](#), for a x16 Standard Package module, the Lane ID for **TD_L[0:15]/RD_L[0:15]** will be represented by the set of Lane ID {0...15} respectively. For a x8 Standard Package module, the Lane ID for **TD_L[0:7]/RD_L[0:7]** will be represented by the set of Lane ID {0...7} respectively.

Table 4-2. Lane ID: Advanced Package module (Sheet 1 of 2)

Logical Lane Number	Lane ID	Logical Lane Number	Lane ID
0	00000000b	34	00100010b
1	00000001b	35	00100011b
2	00000010b	36	00100100b
3	00000011b	37	00100101b
4	00000100b	38	00100110b
5	00000101b	39	00100111b
6	00000110b	40	00101000b
7	00000111b	41	00101001b
8	00001000b	42	00101010b

Table 4-2. Lane ID: Advanced Package module (Sheet 2 of 2)

Logical Lane Number	Lane ID	Logical Lane Number	Lane ID
9	00001001b	43	00101011b
10	00001010b	44	00101100b
11	00001011b	45	00101101b
12	00001100b	46	00101110b
13	00001101b	47	00101111b
14	00001110b	48	00110000b
15	00001111b	49	00110001b
16	00010000b	50	00110010b
17	00010001b	51	00110011b
18	00010010b	52	00110100b
19	00010011b	53	00110101b
20	00010100b	54	00110110b
21	00010101b	55	00110111b
22	00010110b	56	00111000b
23	00010111b	57	00111001b
24	00011000b	58	00111010b
25	00011001b	59	00111011b
26	00011010b	60	00111100b
27	00011011b	61	00111101b
28	00011100b	62	00111110b
29	00011101b	63	00111111b
30	00011110b	64	01000000b
31	00011111b	65	01000001b
32	00100000b	66	01000010b
33	00100001b	67	01000011b

Table 4-3. Lane ID: Standard Package Module

Logical Lane Number	Lane ID	Logical Lane Number	Lane ID
0	00000000b	8	00001000b
1	00000001b	9	00001001b
2	00000010b	10	00001010b
3	00000011b	11	00001011b
4	00000100b	12	00001100b
5	00000101b	13	00001101b
6	00000110b	14	00001110b
7	00000111b	15	00001111b

4.3 Interconnect redundancy remapping

As discussed in [Section 5.9](#), Advanced Package modules require redundancy remapping to recover from faulty Lanes. This section provides the details of remapping. After a successful remapping/repair, any unused repair Lanes must have their Transmitters tri-stated and Receivers disabled.

4.3.1 Data Lane repair

The specification supports remapping (repair) of up to two data Lanes for each group of 32 data Lanes. **TD_P[31:0]** (**RD_P[31:0]**) and **TD_P[63:32]** (**RD_P[63:32]**) are treated as two separate groups of 32 Lanes that can be independently repaired using redundant Lanes, **TRD_P[1:0]** (**RRD_P[1:0]**) and **TRD_P[3:2]** (**RRD_P[3:2]**), respectively. **TD_P[63:32]** (**RD_P[63:32]**) and hence **TRD_P[3:2]** (**RRD_P[3:2]**) do not apply to x32 Advanced Package Link.

Lane remapping is accomplished by “shift left” or “shift right” operation. A “shift left” is when data traffic of logical Lane **TD_L[n]** on **TD_P[n]** is multiplexed onto **TD_P[n-1]**. A shift left puts **TD_L[0]** onto **TRD_P0** or **TD_L[32]** onto **TRD_P2**. A shift right operation is when data traffic **TD_L[n]** is multiplexed onto **TD_P[n+1]**. A shift right puts **TD_L[31]** onto **TRD_P1** or **TD_L[63]** onto **TRD_P3**. See the pseudo codes in [Section 4.3.3.1](#) and [Section 4.3.3.2](#) that show the changes in mapping post repair.

Note: If the lower index redundant Lane (**TRD_P[0]** or **TRD_P[2]**) is faulty, no data lanes can be repaired for its group. Note that if the higher index redundant lane (**TRD_P[1]** or **TRD_P[3]**) is faulty, one data lane can be repaired for its group.

After a data Lane is remapped, the Transmitter associated with the faulty physical Lane is tri-stated and the Receiver is disabled. The Transmitter and the Receiver of the redundant Lane used for the repair are enabled.

[Figure 4-12](#) shows transmit bump side of data Lane remapping for the first group of 32 Lanes. Both “shift left” and “shift right” remapping is needed to optimally repair up to any two Lanes within the group. [Figure 4-13](#) shows details of the mux structure used for data Lane repair.

Note: Example repair implementations are shown for **TD_P[31:0]** for clarity. It should be noted that the same schemes are also applicable to **TD_P[63:32]**.

Figure 4-12. Data Lane remapping possibilities to fix potential defects

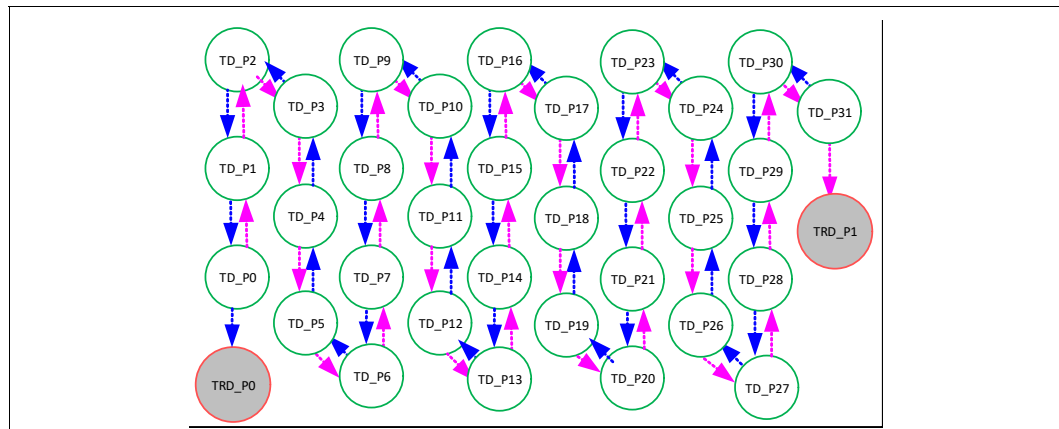
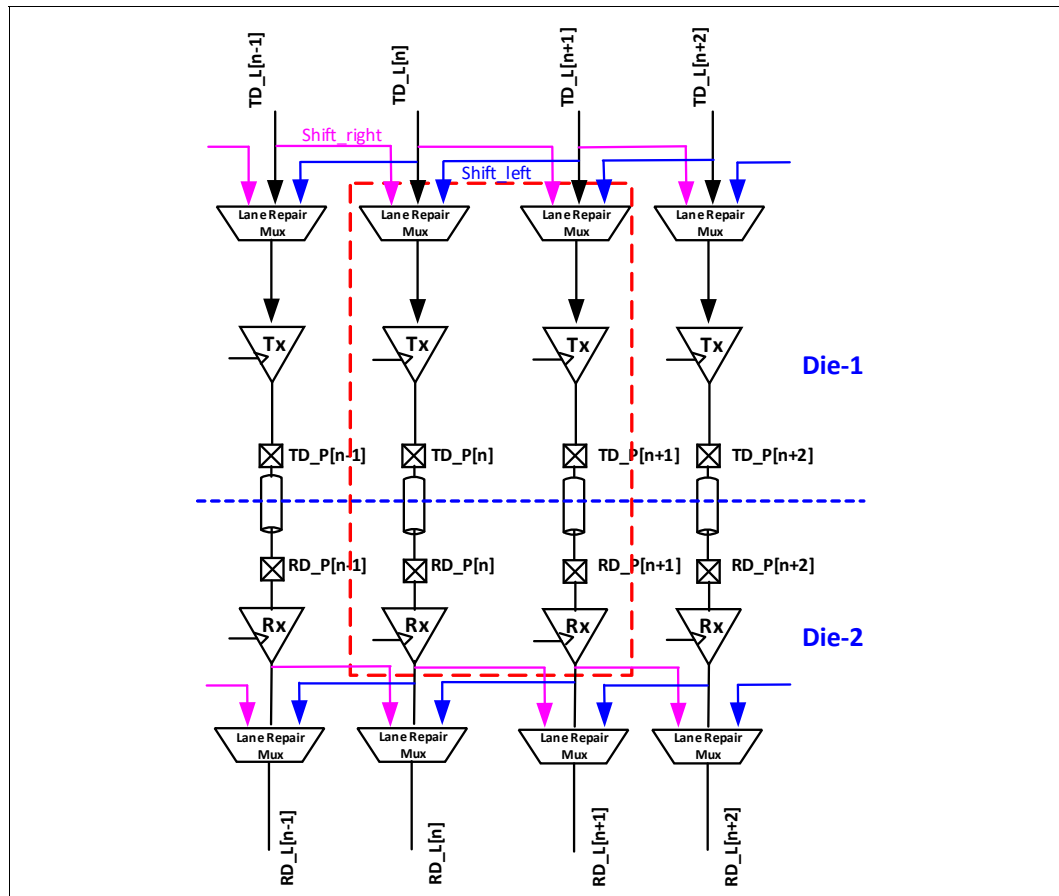


Figure 4-13. Data Lane remapping: Mux chain



4.3.2 Data Lane repair with Lane reversal

If Lanes are reversed, physical Lane 0 of Transmitter (**TD_P[0]**) is connected to physical Lane N-1 of the Receiver (**RD_P[N-1]**). N is 64 or 32 for x64 or x32 Advanced Package modules, respectively. See the pseudo codes in [Section 4.3.3.3](#) and [Section 4.3.3.4](#) that show the changes in mapping post repair.

4.3.3 Data Lane repair implementation

4.3.3.1 Single Lane repair

TRD_P[0] (**RRD_P[0]**) must be used as the redundant Lane to remap any single physical Lane failure for **TD_P[31:0]** (**RD_P[31:0]**). **TRD[2]** (**RRD[2]**) must be used as the redundant Lane to remap any single Lane failure for **TD_P[63:32]** (**RD_P[63:32]**).

Pseudo code for repair in **TD_P[31:0]** (**RD_P[31:0]**) ($0 \leq x \leq 31$):

```
IF failure occurs in TD_P[x]:
    IF x > 0:
        FOR 0 <= i < x:
            TD_P[x-i-1] = TD_L[x-i]
            RD_L[x-i] = RD_P[x-i-1]
        TRD_P[0] = TD_L[0]
        RD_L[0] = RRD_P[0]
```

Pseudo code for repair in **TD_P[63:32]** (**RD_P[63:32]**) ($32 \leq x \leq 63$) (this does not apply to x32 Advanced Package Link):

```
IF failure occurs in TD_P[x]:
    IF x > 32:
        FOR 0 <= i < x-32:
            TD_P[x-i-1] = TD_L[x-i]
            RD_L[x-i] = RD_P[x-i-1]
        TRD_P[2] = TD_L[32]
        RD_L[32] = RRD_P[2]
```

As shown in Figure 4-14 $TD_P[29]$ is remapped in the direction to use $TRD_P[0]$ as the repair resource. Figure 4-15 shows the circuit implementation.

Figure 4-14. Example of Single Lane failure remapping

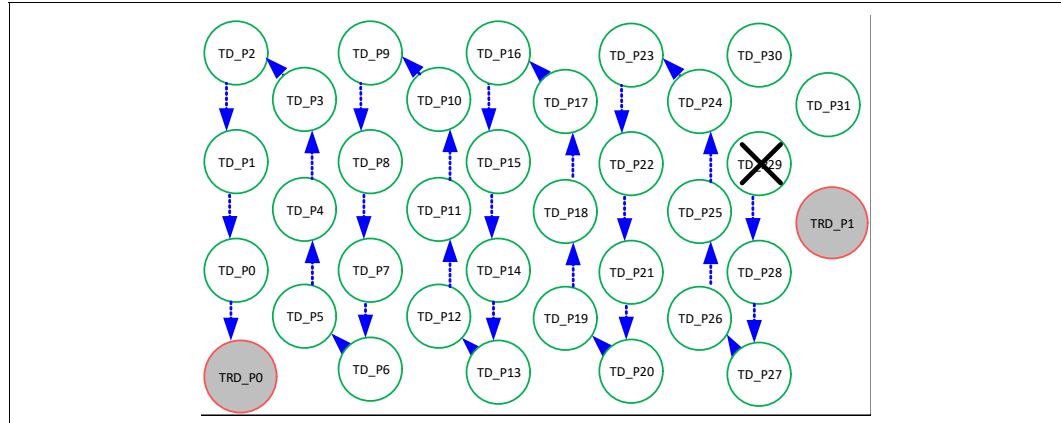
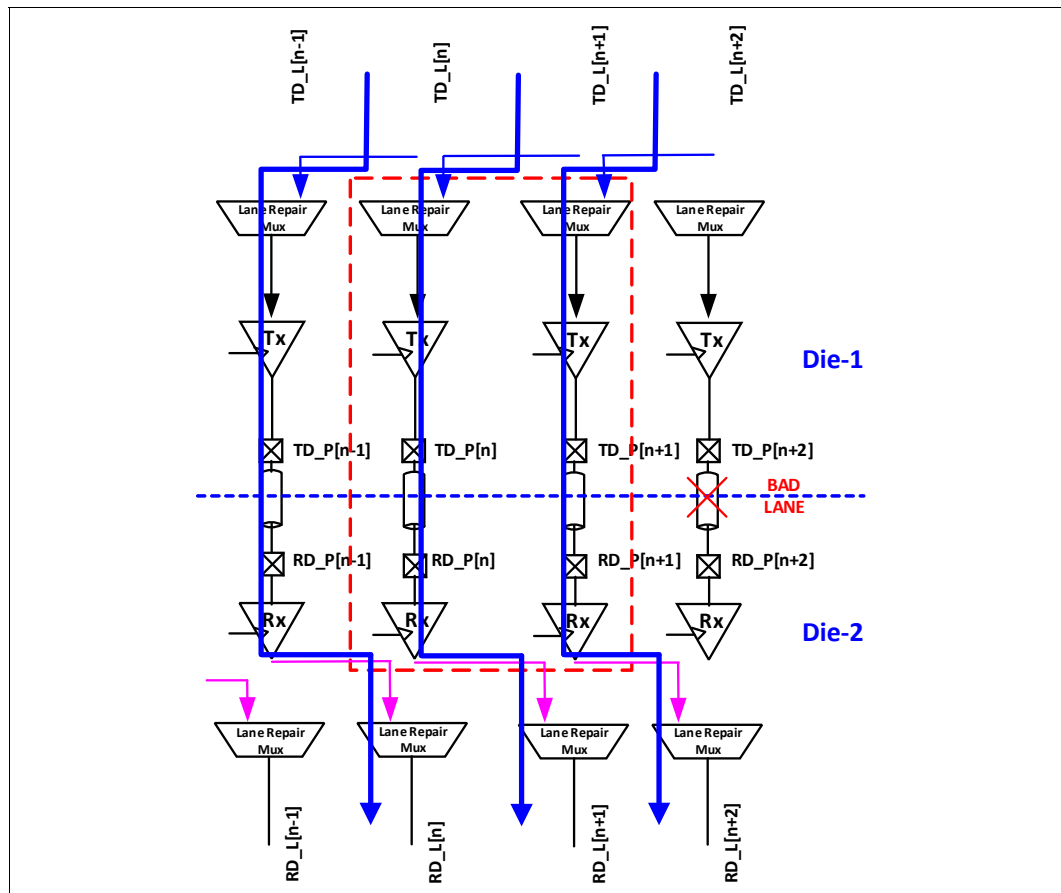


Figure 4-15. Example of Single Lane remapping implementation



4.3.3.2 Two Lane repair

Any two Lanes within a group of 32 can be repaired using the two redundant bumps. For any two physical Lane failures in **TD_P[31:0]** (**RD_P[31:0]**), the lower Lane must be remapped to **TRD_P[0]** (**RRD_P[0]**) and the upper Lane is remapped to **TRD_P[1]** (**RRD_P[1]**). For any two physical Lane failures in **TD_P[63:32]** (**RD_P[63:31]**), the lower Lane must be remapped to **TRD_P[2]** (**RRD_P[2]**) and the upper Lane is remapped to **TRD_P[3]** (**RRD_P[3]**).

Pseudo code for two Lane repair in **TD_P[31:0]** (**RD_P[31:0]**) ($0 \leq x, y \leq 31$):

```

IF failure occurs in TD_P[x], TD_P[y] AND (x < y):
    IF x > 0:
        FOR 0 <= i < x:
            TD_P[x-i-1] = TD_L[x-i]
            RD_L[x-i] = RD_P[x-i-1]
        TRD_P[0] = TD_L[0]
        RD_L[0] = RRD_P[0]
    IF y < 31:
        FOR 0 <= j < (31-y):
            TD_P[y+j+1] = TD_L[y+j]
            RD_L[y+j] = RD_P[y+j+1]
        TRD_P[1] = TD_L[31]
        RD_L[31] = RRD_P[1]

```

Pseudo code for two Lane repair in **TD_P[63:32]** (**RD_P[63:32]**) ($32 \leq x, y \leq 63$) (this does not apply to x32 Advanced Package Link):

```

IF failure occurs in TD_P[x], TD_P[y] AND (x < y):
    IF x > 32:
        FOR 0 <= i < x-32:
            TD_P[x-i-1] = TD_L[x-i]
            RD_L[x-i] = RD_P[x-i-1]
        TRD_P[2] = TD_L[32]
        RD_L[32] = RRD_P[2]
    IF y < 63:
        FOR 0 <= j < (63-y):
            TD_P[y+j+1] = TD_L[y+j]
            RD_L[y+j] = RD_P[y+j+1]
        TRD_P[3] = TD_L[63]
        RD_L[63] = RRD_P[3]

```

Shown in Figure 4-16 is an example of two (physical Lanes 25 and 26) Lane remapping. Figure 4-17 shows the circuit implementation. Both Transmitter and Receiver must apply the required remapping.

Figure 4-16. Example of Two Lane failure remapping

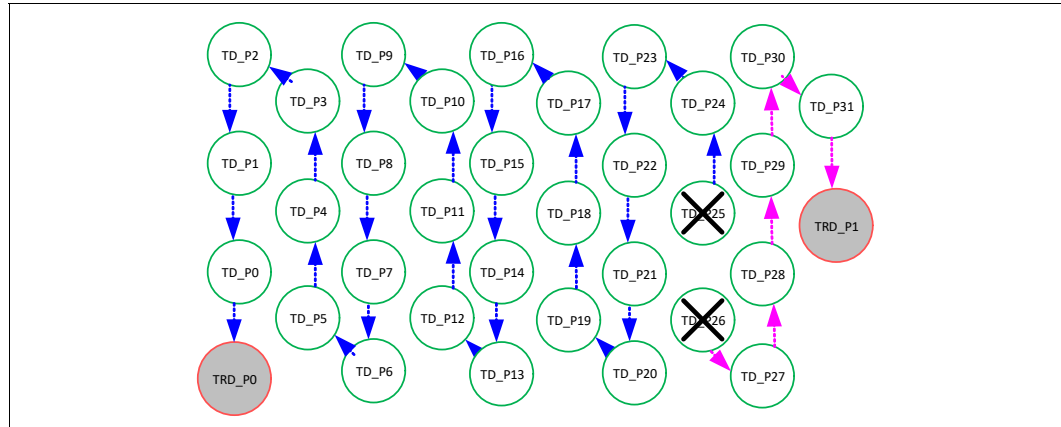
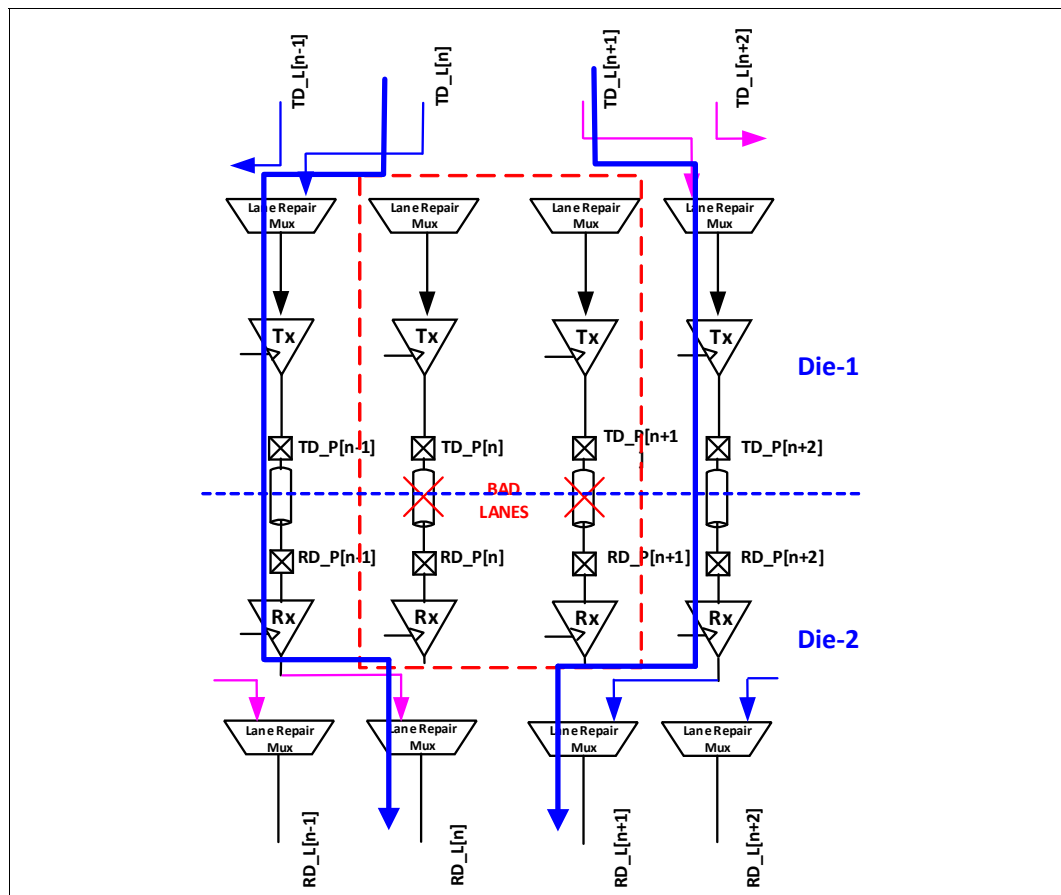


Figure 4-17. Example of Two Lane remapping implementation



4.3.3.3 Single Lane repair with Lane reversal

For repair with Lane reversal (see [Section 4.3.2](#)) Transmitter side remapping is reversed to preserve shifting order for Receiver side remapping.

4.3.3.3.1 x64 Advanced Package Pseudo Codes

Pseudo code for one Lane failure in TD_P[31:0] (RD_P[32:63]) ($0 \leq x \leq 31$):

```

IF failure occurs in TD_P[x]:
    IF x > 0:
        FOR 0 <= i < x:
            TD_P[x-i-1] = TD_L[63-x+i]
            RD_L[63-x+i] = RD_P[63-x+i+1]
        TRD_P[0] = TD_L[63]
        RD_L[63] = RRD_P[3]

```

Pseudo code for one Lane failure in TD_P[63:32] (RD_P[0:31]) ($32 \leq x \leq 63$):

```

IF failure occurs in TD_P[x]:
    IF x > 32:
        FOR 0 <= i < x-32:
            TD_P[x-i-1] = TD_L[63-x+i]
            RD_L[63-x+i] = RD_P[63-x+i+1]
        TRD_P[2] = TD_L[31]
        RD_L[31] = RRD_P[1]

```

4.3.3.3.2 x32 Advanced Package Pseudo Codes

Pseudo code for one-Lane failure in TD_P[31:0] (RD_P[0:31]) ($0 \leq x \leq 31$):

```

IF failure occurs in TD_P[x]:
    IF x > 0:
        FOR 0 <= i < x:
            TD_P[x-i-1] = TD_L[31-x+i]
            RD_L[31-x+i] = RD_P[31-x+i+1]
        TRD_P[0] = TD_L[31]
        RD_L[31] = RRD_P[1]

```

4.3.3.4 Two Lane repair with Lane reversal

For repair with Lane reversal (see [Section 4.3.2](#)) Transmitter side remapping is reversed to preserve shifting order for Receiver side remapping.

4.3.3.4.1 x64 Advanced Package Pseudo Codes

Pseudo code for two Lane failure in TD_P[31:0] (RD_P[32:63]) ($0 \leq x \leq 31$):

```

IF failure occurs in TD_P[x], TD_P[y] AND (x < y):
    IF x > 0:
        FOR 0 <= i < x:
            TD_P[x-i-1] = TD_L[63-x+i]
            RD_L[63-x+i] = RD_P[63-x+i+1]
        TRD_P[0] = TD_L[63]
        RD_L[63] = RRD_P[3]
    IF y < 31:
        FOR 0 <= j < (31-y):
            TD_P[y+j+1] = TD_L[63-y-j]
            RD_L[63-y-j] = RD_P[63-y-(j+1)]
        TRD_P[1] = TD_L[32]
        RD_L[32] = RRD_P[2]

```

Pseudo code for two-Lane failure in TD_P[63:32] (RD_P[0:31]) ($32 \leq x \leq 63$):

```

IF failure occurs in TD_P[x], TD_P[y] AND (x < y):
    IF x > 32:
        FOR 0 <= i < x-32:
            TD_P[x-i-1] = TD_L[63-x+i]
            RD_L[63-x+i] = RD_P[63-x+(i+1)]
        TRD_P[2] = TD_L[31]
        RD_L[31] = RRD_P[1]
    IF y < 63:
        FOR 0 <= j < (63-y):
            TD_P[y+j+1] = TD_L[63-y-j]
            RD_L[63-y-j] = RD_P[63-y-(j+1)]
        TRD_P[3] = TD_L[0]
        RD_L[0] = RRD_P[0]

```


4.3.3.4.2 x32 Advanced Package Pseudo Codes

Pseudo code for two-Lane failure in TD_P[31:0] (RD_P[0:31]) ($0 \leq x \leq 31$):

```

IF failure occurs in TD_P[x], TD_P[y] AND (x < y):
    IF x > 0:
        FOR 0 <= i < x:
            TD_P[x-i-1] = TD_L[31-x+i]
            RD_L[31-x+i] = RD_P[31-x+i+1]
        TRD_P[0] = TD_L[31]
        RD_L[31] = RRD_P[1]
    IF y < 31:
        FOR 0 <= j < (31-y):
            TD_P[y+j+1] = TD_L[31-y-j]
            RD_L[31-y-j] = RD_P[31-y-(j+1)]
        TRD_P[1] = TD_L[0]
        RD_L[0] = RRD_P[0]

```

4.3.4 Clock and Track Lane remapping

The specification supports remapping of one broken Lane for **TCKP_P/RCKP_P**, **TCKN_P/RCKN_P** and **TTRK_P/RTRK_P** physical Lanes. The repair scheme is shown in Figure 4-18. Clock Lane remapping allows repair of single Lane failure for both differential and pseudo-differential implementation of the clock Receiver. The circuit details are shown in Figure 4-19 and Figure 4-20 for differential and pseudo-differential clock Receivers respectively.

After a Lane is remapped, the Transmitter is tri-stated. The Receiver of the physical redundant (**RRDCK_P**) Lane is disabled.

Figure 4-18. Clock and Track repair

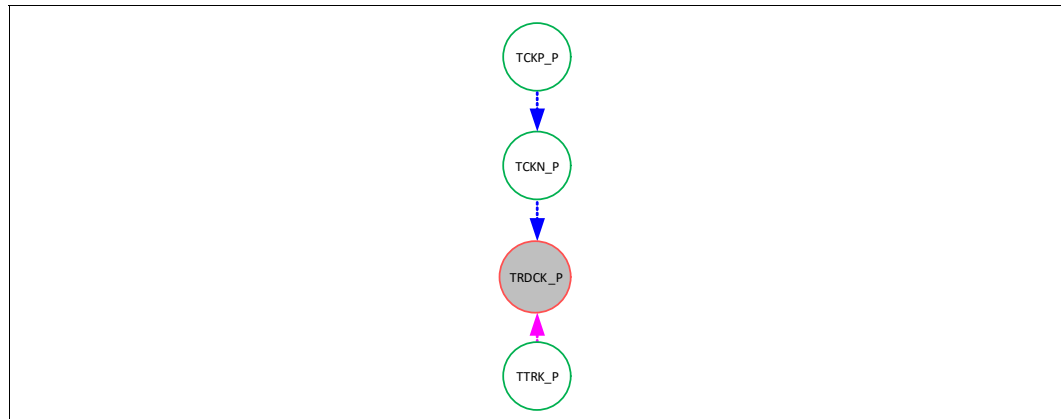


Figure 4-19. Clock and track repair: Differential Rx

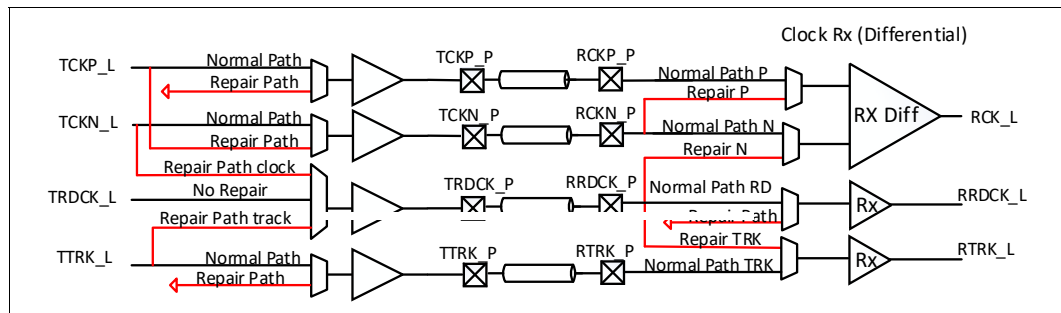
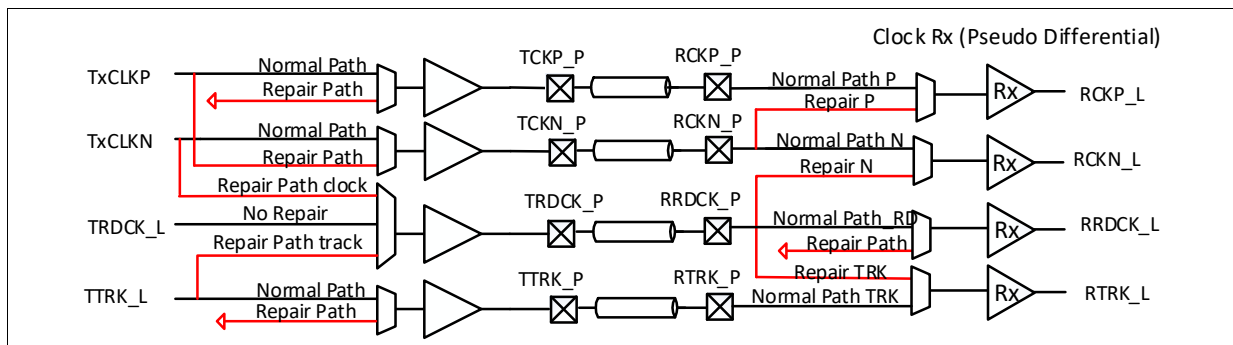


Figure 4-20. Clock and track repair: Pseudo Differential Rx



4.3.5 Clock and Track Lane repair implementation

Pseudo code for Clock and Track Lane repair:

```

IF failure occurs in TCKP_P:
    TCKN_P = TCKP_L AND TRDCK_P = TCKN_L
    RCKP_L = RCKN_P AND RCKN_L = RRDCK_P
ELSE IF failure occurs on TCKN_P:
    TRDCK_P = TCKN_L
    RCKN_L = RRDCK_P
ELSE IF failure occurs in TTRK_P:
    TRDCK_P = TTRK_L
    RTRK_L = RRDCK_P

```

The implementation of Clock and Track Lane remapping is shown in Figure 4-21(a), Figure 4-21(b) and Figure 4-21(c) respectively. The corresponding circuit level details of remapping implementation are shown in Figure 4-22, Figure 4-23 and Figure 4-24.

Note that the both Transmitter and Receiver on CKRD Lane are required during detection phase and can be tri-stated and turned off if not used for repair.

Figure 4-21. Clock and Track Lane repair scheme

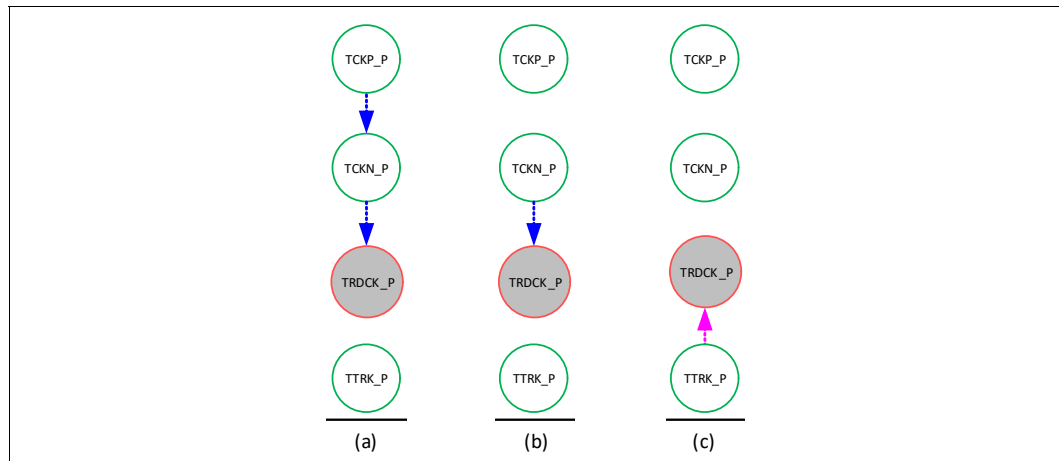


Figure 4-22. Clock and track repair: CKP repair

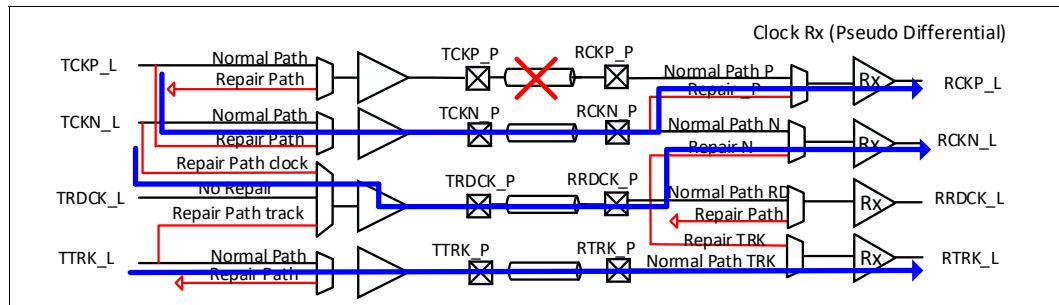


Figure 4-23. Clock and track repair: CKN repair

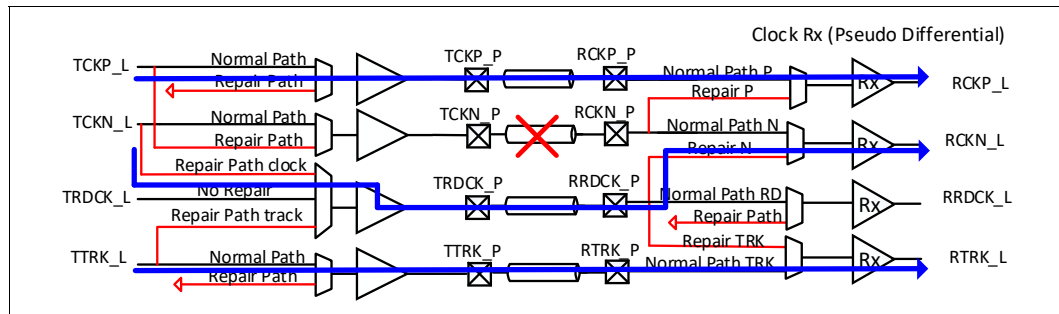
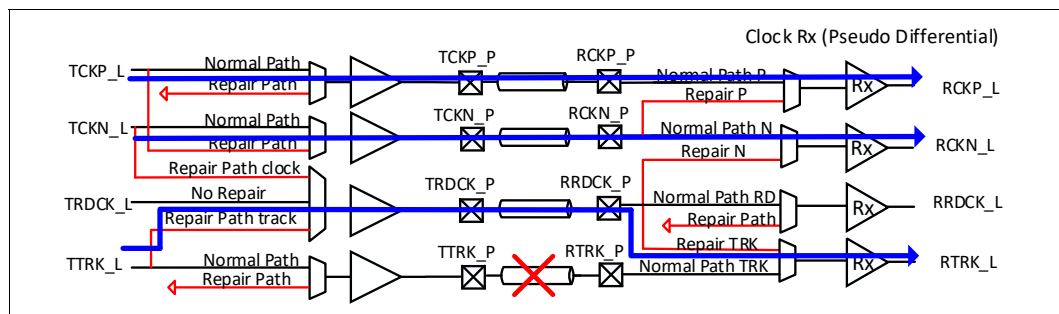


Figure 4-24. Clock and track repair: Track repair



4.3.6 Valid Repair and implementation

Valid Lane has a dedicated redundant Lane. If a failure is detected on the Valid physical Lane, redundant Valid physical Lane is used to send Valid. Valid failure detection and repair is performed during Link initialization and Training (Section 4.5.3.3.4).

Figure 4-25 shows the normal path for Valid and redundant valid Lanes. Figure 4-26 shows the repair path for Valid Lane failure.

Figure 4-25. Valid repair: Normal Path

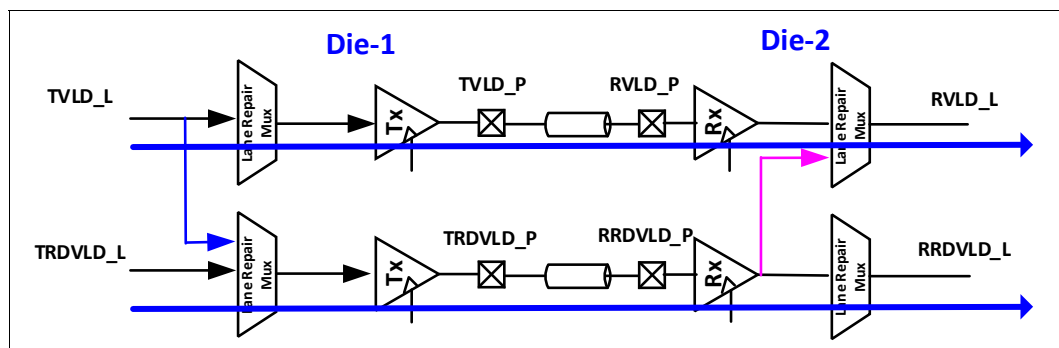
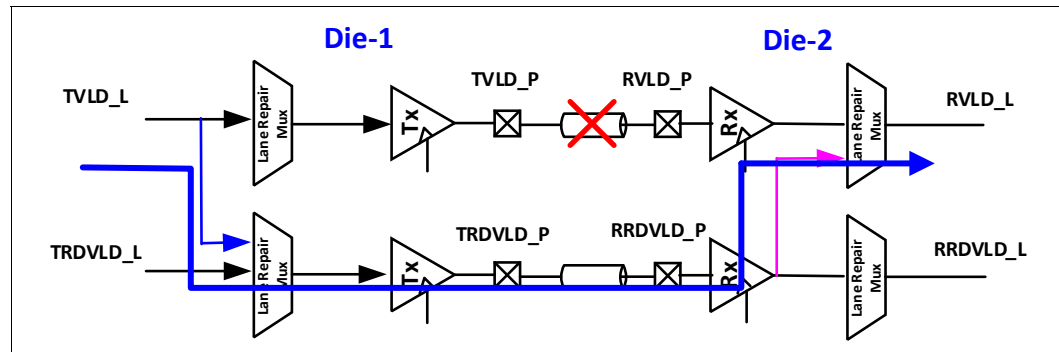


Figure 4-26. Valid Repair: Repair Path



4.3.7 Width Degrade in Standard Package Interfaces

In the case of x16 Standard Package modules where Lane repair is not supported, resilience against faulty Lanes is provided by configuring the Link to a x8 width (Logical Lanes 0 to 7 or Logical Lanes 8 to 15, which exclude the faulty Lanes). For example, if one or more faulty Lanes are in logical Lane 0 to 7, the Link is configured to x8 width using logical Lanes 8 to 15. The configuration is done during Link initialization or retraining. Transmitters of the disabled Lanes are tri-stated and Receivers are disabled.

In the case of x8 Standard Package modules, resilience against faulty Lanes is provided by configuring the Link to a x4 width (Logical Lanes 0 to 3 or Logical Lanes 4 to 7, which exclude the faulty Lanes). The configuration is done during Link initialization or retraining. Transmitters of the disabled Lanes are tri-stated and Receivers are disabled.

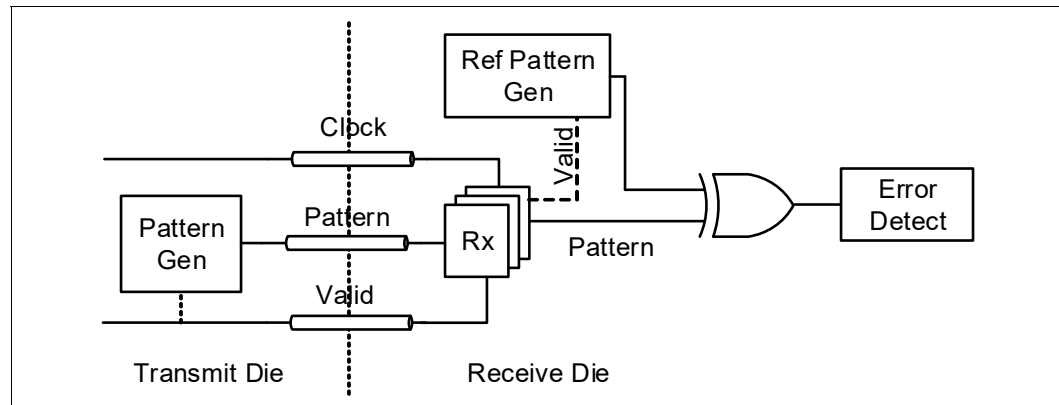
Figure 4-5 shows the byte to Lane mapping for a width degraded x8 interface

4.4 Data to Clock Training and Test Modes

Note: Sideband commands will be identified as {SB command}.

Figure 4-27 shows the infrastructure for interface training and testing. The Transmit Die and Receive Die implement the same Linear Feedback Shift Register (LFSR) described in Section 4.4.1. The pattern sent from the Transmitter along with forwarded clock and Valid is compared with locally generated reference pattern. Both transmit and receive pattern generators must start and advance in sync. The compare circuitry checks for matching data each UI. Any mismatch between the received pattern and pattern predicted by the local pattern generator is detected as an error.

Figure 4-27. Test and training logic



The receive die must implement two types of comparison schemes

- Per-Lane comparison:** Per-Lane comparison is used to identify the number of failing Lanes between the two dies. Any mismatch, above the set threshold between the received pattern and the expected pattern on each Lane, will set an internal error detect bit for each Lane. Once a pattern mismatch on a particular Lane is found, this error bit is set for the remainder of the test. [Figure 4-28](#) illustrates Per-Lane comparison mode. This mode will indicate per-Lane errors within the module ($N = 68$ ($64 + 4$ RD) for a x64 Advanced Package Module, $N = 34$ ($32 + 2$ RD) for a x32 Advanced Package Module, $N = 16$ for a x16 Standard Package Module, and $N = 8$ for a x8 Standard Package Module, respectively). The per-Lane comparison results can be read via sideband.
- Aggregate comparison:** In this mode, pattern mismatches each UI on any Lane within the module are accumulated into a 16-bit error counter. The Lane errors are ORED to generate a module-level error and counted as shown in [Figure 4-29](#). This scheme can be used for module level margin and BER.

Figure 4-28. Lane failure detection

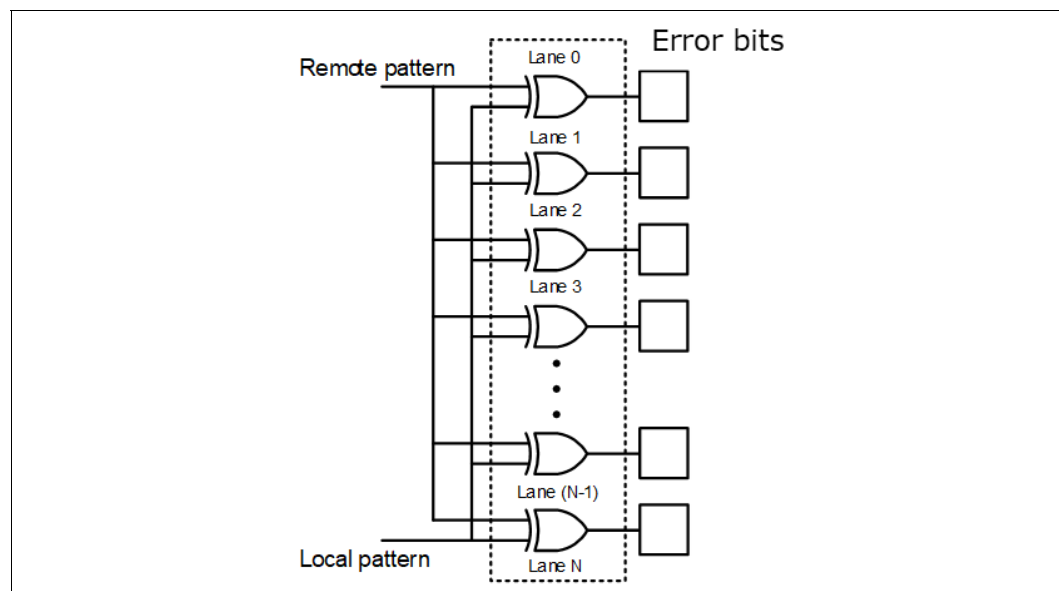
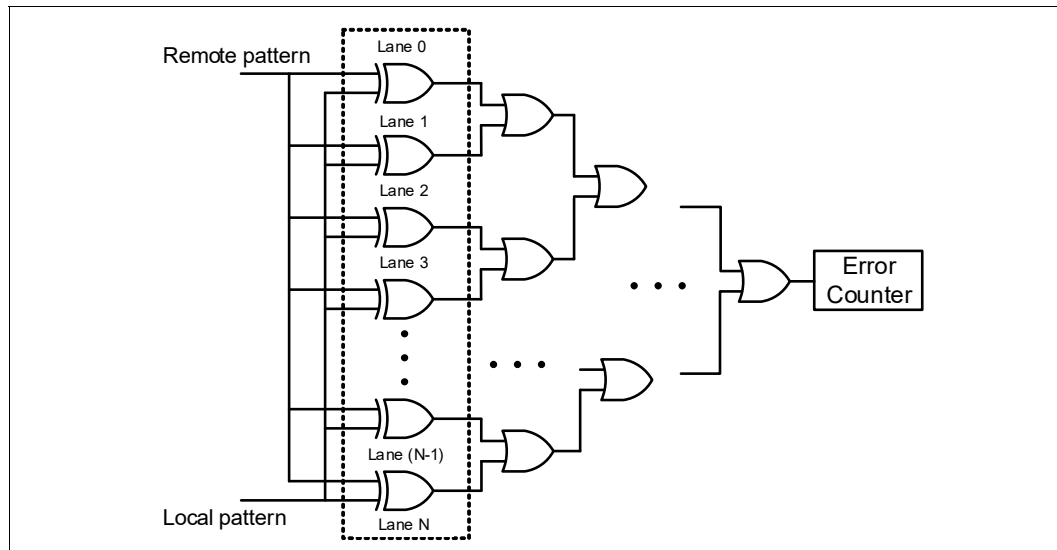


Figure 4-29. All Lane error detection



4.4.1 Scrambling and training pattern generation

A Linear feedback shift register (LFSR) is defined for scrambling and test pattern generation.

The LFSR uses the same polynomial as PCIe: $G(X) = X^{23} + X^{21} + X^{16} + X^8 + X^5 + X^2 + 1$. Each Transmitter is permitted to implement a separate LFSR for scrambling and pattern generation. Each Receiver is permitted to implement a separate LFSR using the same polynomial for de-scrambling and pattern comparison. The implementation is shown in Figure 4-30. The seed of the LFSR is Lane dependent, and based on the Logical Lane number and the seed value for Lane number is modulo 8 as shown in Table 4-4.

Alternatively, implementations can choose to implement one LFSR with different tap points for multiple Lanes as shown in Figure 4-31. This is equivalent to individual LFSR per-Lane with different seeds.

Table 4-4. LFSR seed values

Lane	Seed
0	23'h1DBFBC
1	23'h 0607BB
2	23'h1EC760
3	23'h18C0DB
4	23'h010F12
5	23'h19CFC9
6	23'h0277CE
7	23'h1BB807
RD0, RD2 ^a	23'h18C0DB
RD1, RD3 ^b	23'h010F12

a. Same as Lane 3. These are not currently used.

b. Same as Lane 4. These are not currently used.

Figure 4-30. LFSR implementation

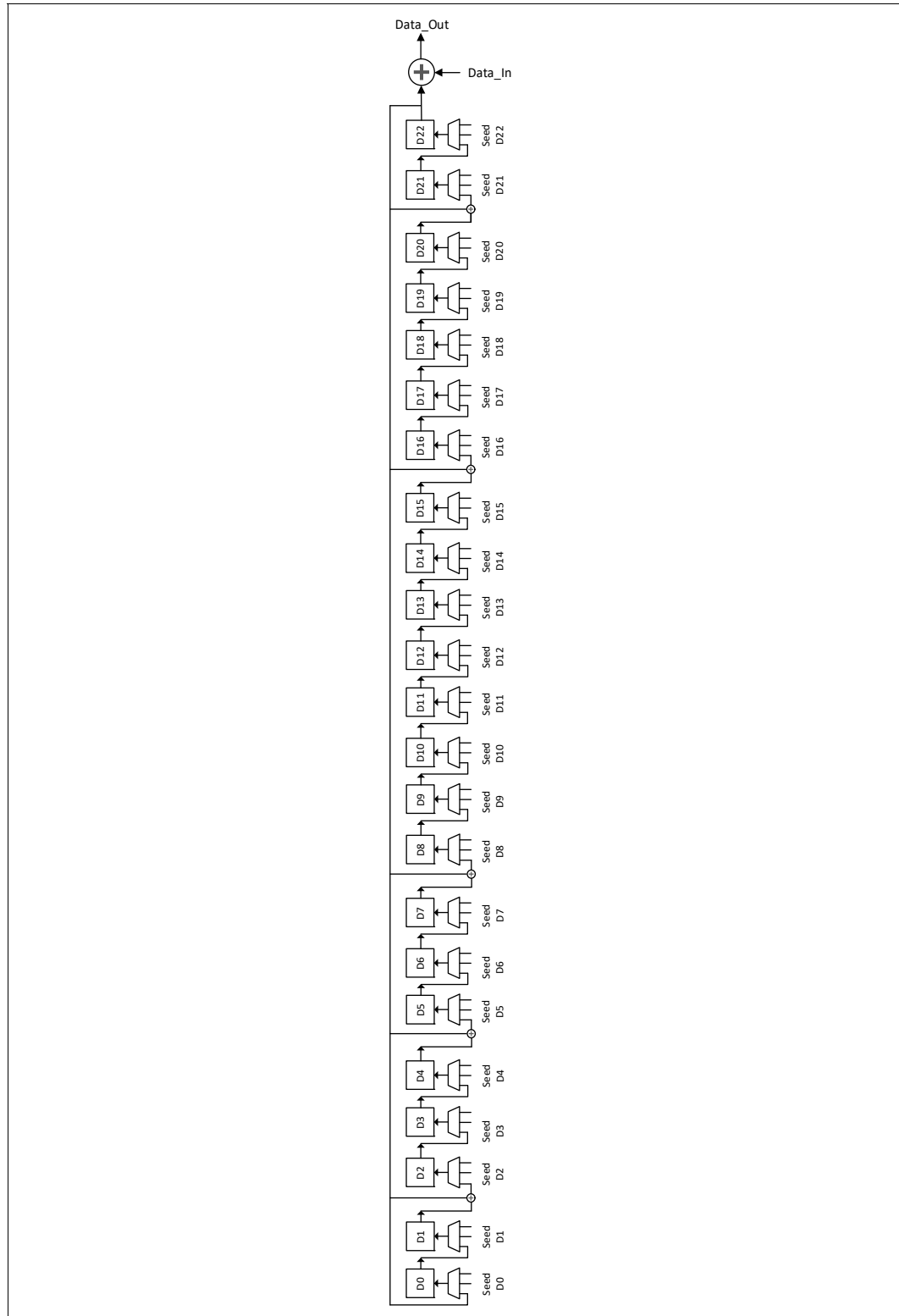
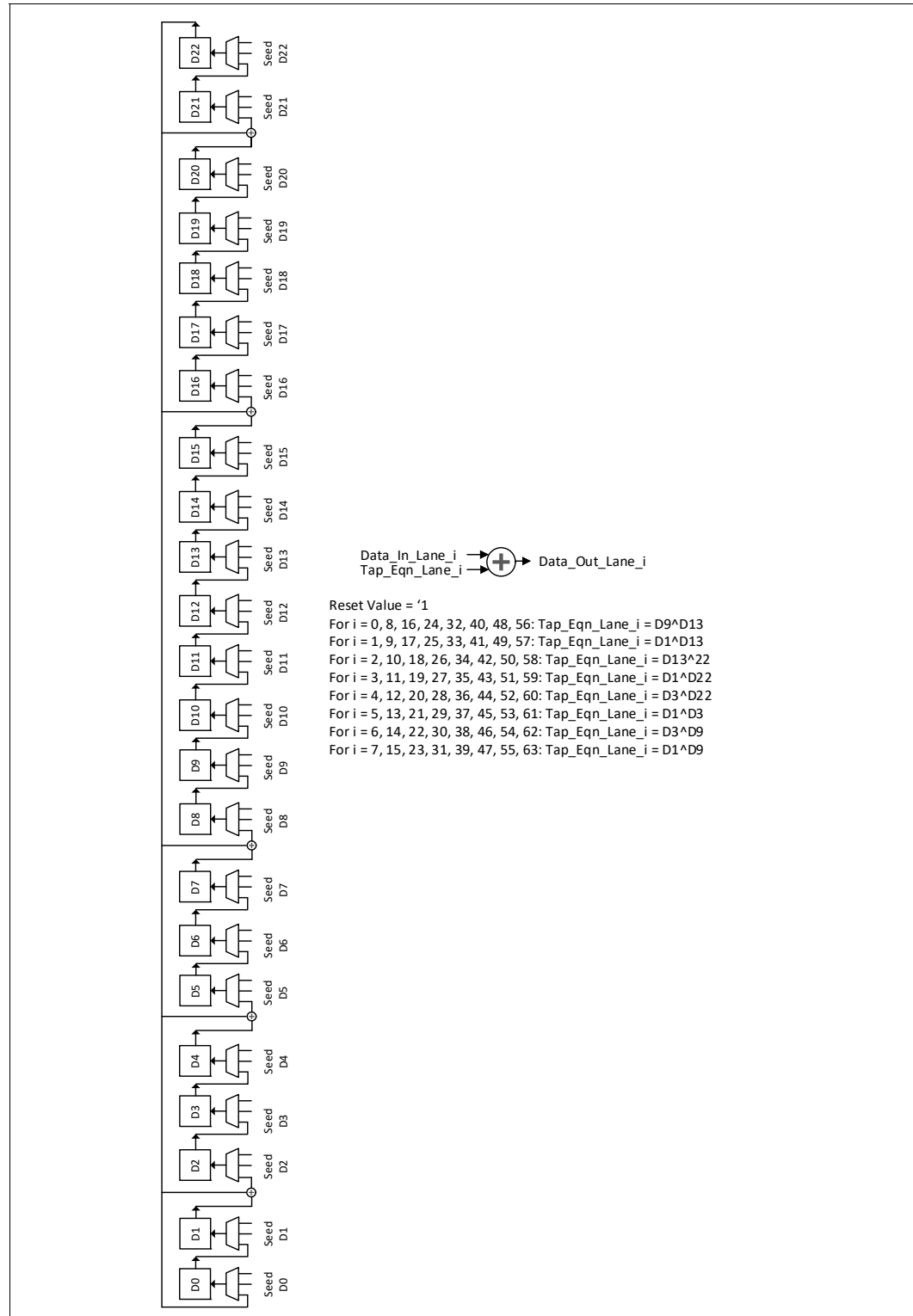


Figure 4-31. LFSR alternate implementation



4.5 Link Initialization and Training

Link initialization and training are described at a Module level. The description of terms are used in this section are as follows:

- **UCIe Module Partner:** A UCIe Module Partner is the corresponding UCIe Module on the remote UCIe Die to which the UCIe Module connects. For example, if two UCIe Dies A and B are connected in the standard Die rotate configuration by a 2-module UCIe Link, Modules 0 and 1; UCIe Die A's Module 0 (UCIe Module A0) is connected to UCIe Die B's Module 1 (UCIe Module B1); then A0 is the UCIe Module Partner of B1 and vice-versa.
- **Phase Interpolator (PI)** in this specification is used to refer any method of generating and selecting sampling clock phase.
- **Timeout:** Every state except RESET, Active, L1/L2, and TRAINERROR in the Link Training State Machine has a residency timeout of 8 ms. For states with sub-states, the timeout is per sub-state (i.e., if Physical Layer is in a state for greater than 8 ms, then it transitions to TRAINERROR and eventually to RESET). Physical Layer sideband handshakes for RDI state transitions with remote Link partner also timeout after 8 ms. The timeout counters must be reset if a sideband message with "Stall" encoding is received. All timeout values specified are -0% and +50% unless explicitly stated otherwise. All timeout values must be set to the specified values after Domain Reset exit. All counter values must be set to the specified values after Domain Reset.
- Electrical idle is described in [Section 5.12](#).
- When Management Transport protocol is supported over the sideband, the term SB_MGMT_UP refers to an internally maintained flag in the PHY that indicates whether the Management Transport path has been successfully initialized with the partner chiplet. If the flag is cleared to 0, the Management Transport path is not up. If the flag is set to 1, the Management Transport path is up. The flag is cleared on a Management Reset or when a Heartbeat timeout is detected (see [Section 8.2.5.1.3](#)). The flag is set after successful completion of the initialization phase of Management Transport path setup as described in [Section 8.2.3.1.2](#).

When training during Link Initialization (i.e., Physical Layer transitions out of RESET state), hardware is permitted to attempt training multiple times:

- Triggers for initiating Link Training for Management Transport path setup on the sideband are:
 - Software writes 1 to the Retrain Link bit in the Sideband Management Port Structure register (see [Section 8.1.3.6.2.1](#))
 - HW-autonomous trigger by the Management Port Gateway to automatically train the Management Transport path on the sideband without SW intervention, as occurs after Management Reset
 - SBINIT pattern (two consecutive iterations of 64-UI clock pattern and 32-UI low) is observed on any sideband Receiver clock/data pair, when the SB_MGMT_UP flag is cleared to 0
- The triggers for initiating Link Training for the mainband are:
 - Software writes 1 to Start UCIe Link Training bit in UCIe Link Control register in the UCIe Link DVSEC (see [Section 9.5.1.5](#))
 - When Management Transport protocol is supported on the UCIe link, software writes 1 to the Retrain Link bit in the Management Port Structure register of either the Sideband or Mainband Management Port that is associated with the UCIe link (see [Section 8.1.3.6.2.1](#))
 - Adapter triggers Link Training on the RDI (RDI status is Reset and there is a NOP to Active transition on the state request)

- If [Management Transport protocol is not supported] OR [the SB_MGMT_UP flag is cleared to 0], the SBINIT pattern (two consecutive iterations of 64-UI clock pattern and 32-UI low) is observed on any sideband Receiver clock/data pair
- If [Management Transport protocol is supported] AND [the SB_MGMT_UP flag is set to 1], the {SBINIT done req} sideband message was received from the module partner

If hardware fails training after an implementation-specific number of attempts, the hardware must transition to RESET and wait for a subsequent Link Training trigger. Physical Layer must escalate a fatal error to the D2D Adapter on the RDI if mainband Software-triggered or RDI-triggered Link training fails or there is a Link-up-to-Link-down transition due to a Physical Layer timeout.

Throughout this section, references to mainband transmitter and receiver behavior are called out in various state machine states. These references do not apply when the port does not have a mainband (i.e., the port is a sideband-only port without a physically present mainband, or the port is a UCIE-S port with only the sideband used. In the latter scenario, the mainband transmitters are held in tri-state throughout and the mainband receivers are disabled.

4.5.1 Link Training Basic Operations

Some basic operations in Link training are defined in this section. These will be used in mainband initialization, training and margining.

4.5.1.1 Transmitter initiated Data to Clock Point Test

In this mode, the Transmitter initiates the data to clock training on all Lanes in the module at a single PI phase. When not performing the actions relevant to this state:

- Data, Valid, and Track Transmitters drive low
- Clock Transmitters are held differential low (for differential clocking) or simultaneous low (for Quadrature clocking)
- Data, Valid, and Clock Receivers are enabled
- Track Receiver is permitted to be disabled

The sequence of steps for this test are as follows for each UCIE Module of the UCIE Link:

1. The UCIE Module sets up the Transmitter parameters (shown in [Table 4-5](#)), sends a {Start Tx Init D to C point test req} sideband message to its UCIE Module Partner, and waits for a response. The data field of this message includes the required parameters, shown in [Table 4-5](#). The Receiver on the UCIE Module Partner must enable the pattern comparison circuits to compare incoming mainband data to the locally generated expected pattern. Once the data to clock training parameters for its Receiver are setup, the UCIE Module Partner responds with a {Start Tx Init D to C point test resp} sideband message.
2. The UCIE Module resets the LFSR (scrambler) on its mainband Transmitters and sends the {LFSR clear error req} sideband message. The UCIE Module Partner resets the LFSR and clears any prior compare results on its mainband Receivers and responds with {LFSR clear error resp} sideband message.
3. The UCIE Module sends the pattern (selected through "Tx Pattern Generator Setup") for the selected number of cycles ("Tx Pattern Mode Setup") on its mainband Transmitter.
4. The UCIE Module Partner performs the comparison on its Receivers for each UI during the pattern transmission based on "Rx compare setup" and logs the results.